

Reinforcement Learning

Bing-Yi JING
荆炳义

majing@ust.hk

Agenda

- Dynamic programming: full MDP
- Policy Evaluation (Prediction): $v_\pi(s)$ evaluate policy
- Policy Improvement (Policy Iteration):
  $q_\pi(s, \pi'(s)) \geq v_\pi(s)$, and $\pi_0 \rightarrow \pi_1 \rightarrow \pi_2, \ldots$
- Value Iteration
- Synchronous / Asynchronous Dynamic Programming
- Generalized Policy Iteration

Dynamic Programming

What is Dynamic Programming (DP)?

- Use value function $v_\pi(s)$ to search for a good policy π
- Full knowledge of the MDP, used for planning
- Planning: policy evaluation & policy improvement

Planning by DP

Prediction

- Input: $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ & π (MDP & Policy)
- Output: v_π evaluate (value function)

Control

- Input: $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ (MDP)
- Output: v_* & π_* (optimal value & optimal policy)

Planning by DP

Prediction

- Input: $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ & π (MDP & Policy)
- Output: v_π (value function)

Control

- Input: $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ (MDP)
- Output: v_* & π_* (optimal value & optimal policy)

$\pi(a \mid s)$ is under our control, and to be optimised!

Applications of DP

- Scheduling algorithms
- String algorithms (e.g. sequence alignment)
- Graph algorithms (e.g. shortest path algorithms)
- Graphical models (e.g. Viterbi algorithm)
- Bioinformatics (e.g. lattice models)

Policy evaluation

Prediction: $v_\pi(s) = \mathbf{E}_\pi(G_t \mid S_t = s)$

Recall: Bellman equation for MDPs

The state-value function can again be decomposed into immediate reward plus discounted value of successor state,

$$v_\pi(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s \right]$$

The action-value function can similarly be decomposed,

$$q_\pi(s, a) = \mathbb{E}_\pi \left[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a \right]$$

Equivalently

$$v_\pi(s) = \mathbf{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \,|\, S_t = s]$$

$$= \sum_a \pi(a \,|\, s) \sum_{s', r} p(s', r \,|\, s, a)[r + \gamma v_\pi(s')]$$

$$(\forall s \in \mathcal{S})$$

Existence and uniqueness: guaranteed for $\gamma < 1$

Solving Bellman directly

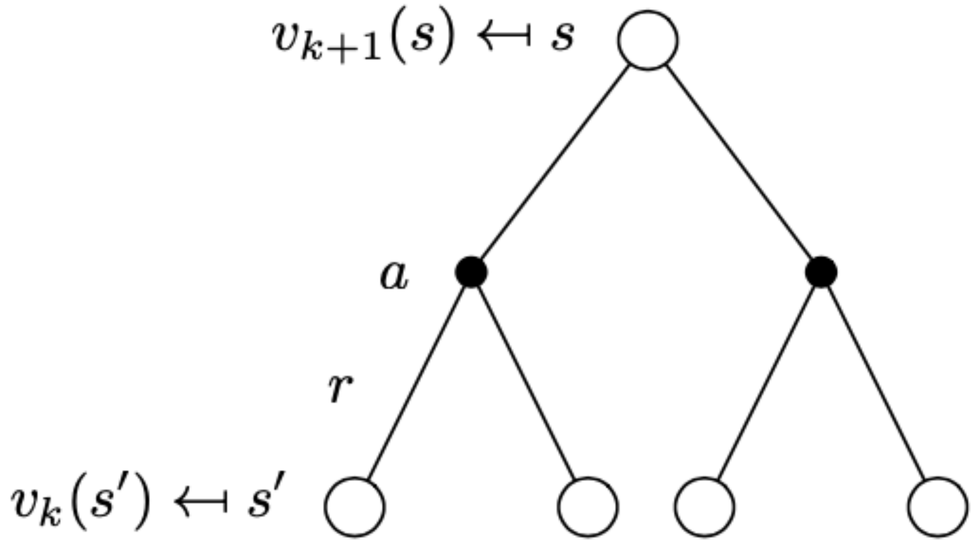
- A system of $|\mathcal{S}|$ simultaneous linear equations
- $|\mathcal{S}|$ unknowns: $v_\pi(s),\ s \in \mathcal{S}$
- Direct solution: straightforward, but tedious

Iterative policy evaluation

$$v_{k+1}(s) := \mathbf{E}_\pi[R_{t+1} + \gamma v_k(S_{t+1}) \,|\, S_t = s]$$

$$= \sum_a \pi(a \,|\, s) \sum_{s',r} p(s', r \,|\, s, a)[r + \gamma v_k(s')]$$

$$(\forall s \in \mathcal{S})$$

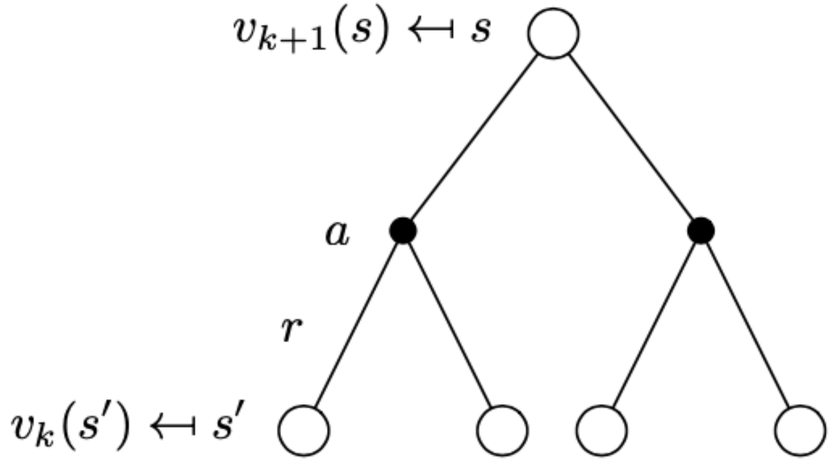


i.e.

$$\mathbf{v}_{k+1} = \mathscr{R}^\pi + \gamma \mathscr{P}^\pi \mathbf{v}_k$$

Iterative policy evaluation

- Problem: evaluate a given policy π
- Solution: iterative application of Bellman expectation backup
- $v_1 \rightarrow v_2 \rightarrow ... \rightarrow v_\pi$
- Using *synchronous* backups,
  - At each iteration $k+1$
  - For all states $s \in \mathcal{S}$
  - Update $v_{k+1}(s)$ from $v_k(s')$
  - where s' is a successor state of s
- We will discuss *asynchronous* backups later
- Convergence to v_π will be proven at the end of the lecture



Algorithm

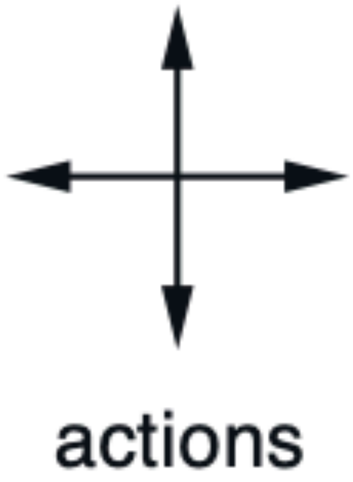
Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π, the policy to be evaluated
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\mathit{terminal}) = 0$

Loop:
    $\Delta \leftarrow 0$
    Loop for each $s \in \mathcal{S}$:
        $v \leftarrow V(s)$
        $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\left[r + \gamma V(s')\right]$
        $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Example: gridworld

Consider the 4×4 gridworld shown below.



actions

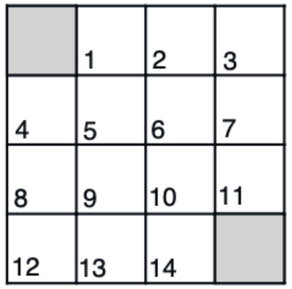
	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$$R_t = -1$$
on all transitions

Example: gridworld

Consider the 4×4 gridworld shown below.





$R_t = -1$
on all transitions

- $\mathcal{S} = \{1,...,14\}$ + 2 grey regions
- (14 non-terminal + 2 terminal states)
- $\mathcal{A} = \{\text{up, down, left, right}\}$
- $\mathcal{R}$: $r(s, \cdot) = -1, \forall s \neq Grey$; and 0 otherwise.
- $\mathcal{P}$: e.g., $\mathbf{P}(Grey \mid 1, left) = 1; \mathbf{P}(11 \mid 10, right) = 1$, $\mathbf{P}(13 \mid 13, down) = 1$
  (i.e. actions leading out of grid leave state unchanged)
- $\gamma = 1$, undiscounted episodic MDP
- $\pi(a \mid s) = 1/4$, $\forall a \in \mathcal{A}$, $s = 1,...,14$ [i.e., random policy]

v_k for the random policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

Demo: $v_{k+1}(s') = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_k(s')]$

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

Initial value: $v_0(s) = 0$

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$$v_1(Grey) = 0 + 0 = 0$$
$$v_1(1) = -1 + 0 = -1$$
$$\cdots\cdots$$
$$v_1(14) = -1 + 0 = -1$$

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$$v_2(Grey) = 0 + 0 = 0$$
$$v_2(1) = -1 + (1/4) * (0 - 1 - 1 - 1) = -7/4 = -1.75$$
$$v_2(2) = -1 + (1/4) * (-1 - 1 - 1 - 1) = -1 - 1 = -2$$
$$\cdots\cdots$$
$$v_2(14) = -1 + (1/4) * (-1 - 1 - 1 + 0) = -1 - 3/4 = -1.75$$

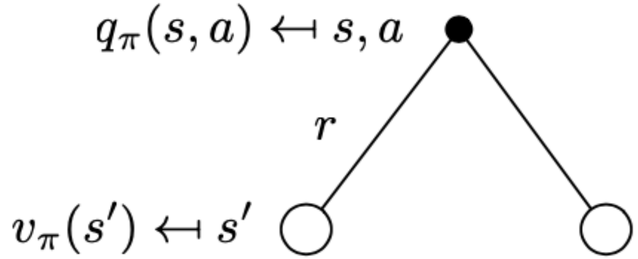
Exercise: verify v_k yourself on last page

Policy Improvement

How to improve policy?

Explore different actions to seek better policy

- The action-value function $q_\pi(s, a)$ is the expected return startin from state s, taking action a, and then following policy π

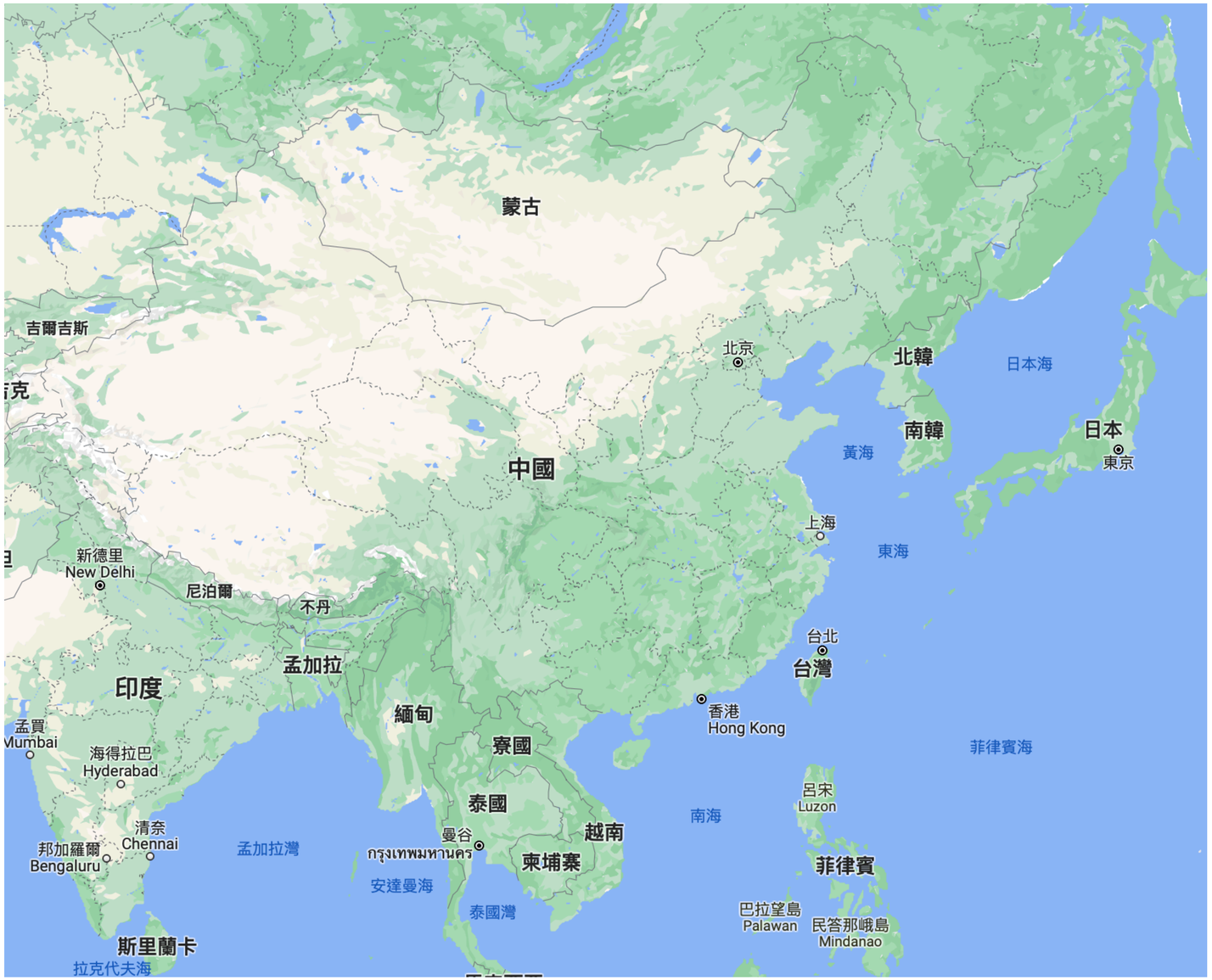


$$q_\pi(s, a) = r_\pi(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a)\ v_\pi(s')$$

$$q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s')$$

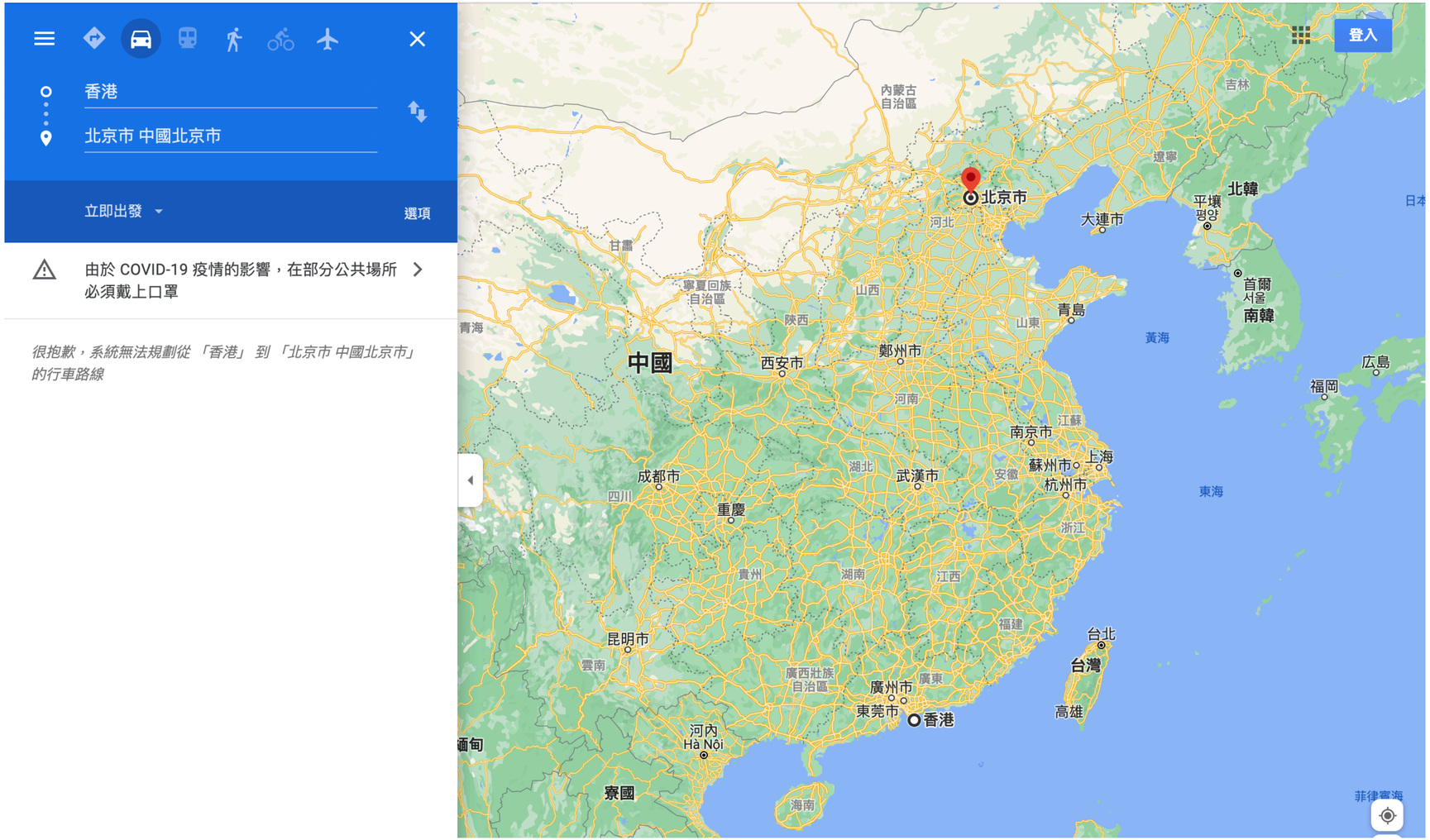
- If $q_\pi(s, a) \geq v_\pi(s)$, then it is better selecting a in s, and thereafter following π, resulting in an improved policy

Driving from HK to Beijing?



HK to Beijing

Forget it


香港
北京市 中國北京市
立即出發
選項
由於 COVID-19 疫情的影響，在部分公共場所必須戴上口罩
很抱歉，系統無法規劃從 「香港」 到 「北京市 中國北京市」的行車路線
登入
北京市
中國
香港
台北
台灣
高雄
北韓
南韓
首爾
서울
平壤
평양
大連市
青島
鄭州市
西安市
南京市
上海
蘇州市
杭州市
武漢市
成都市
重慶
昆明市
廣州市
東莞市
河內
Hà Nội
寮國
緬甸
內蒙古自治區
吉林
遼寧
河北
山西
山東
陝西
甘肅
寧夏回族自治區
青海
河南
江蘇
安徽
湖北
湖南
江西
浙江
福建
廣東
四川
貴州
雲南
廣西壯族自治區
海南
黃海
東海
日本
広島
福岡

Driving from Shenzhen to Beijing



SZ to Beijing

Many choices

Objectives?

fastest, or

most scenic

Day 0



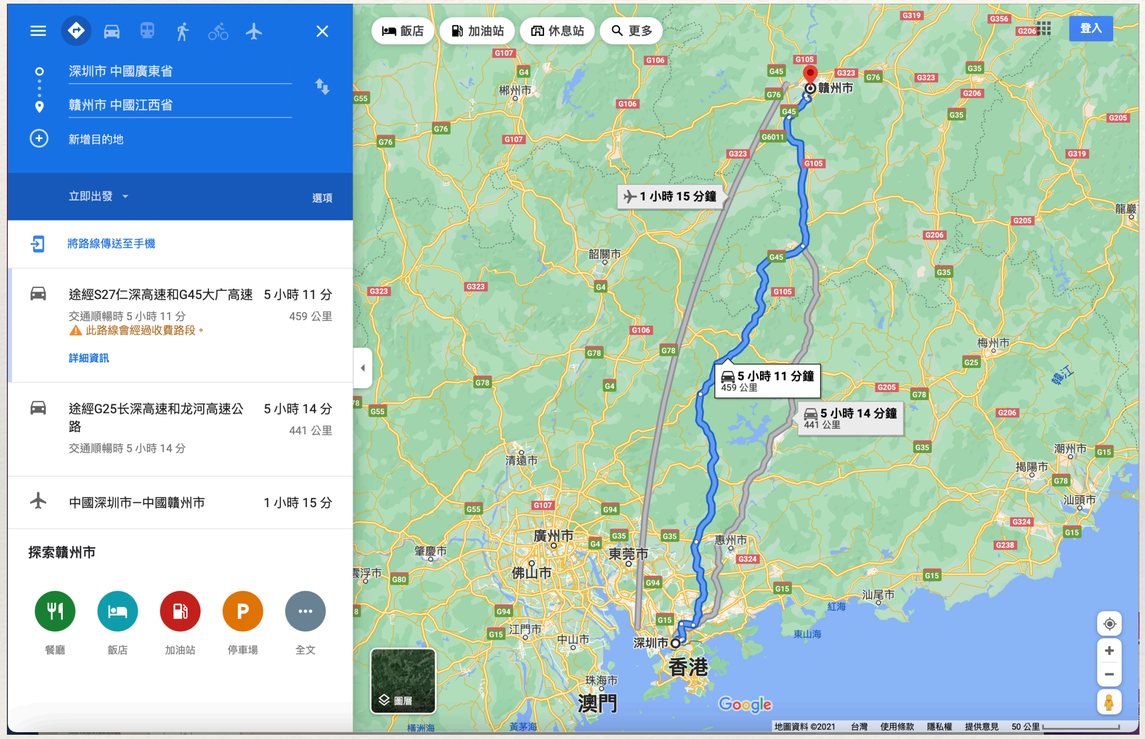
Estimated travel time:
22 hours 26 mins

(22h 3m + 22h 49m)/2
=22h 26m

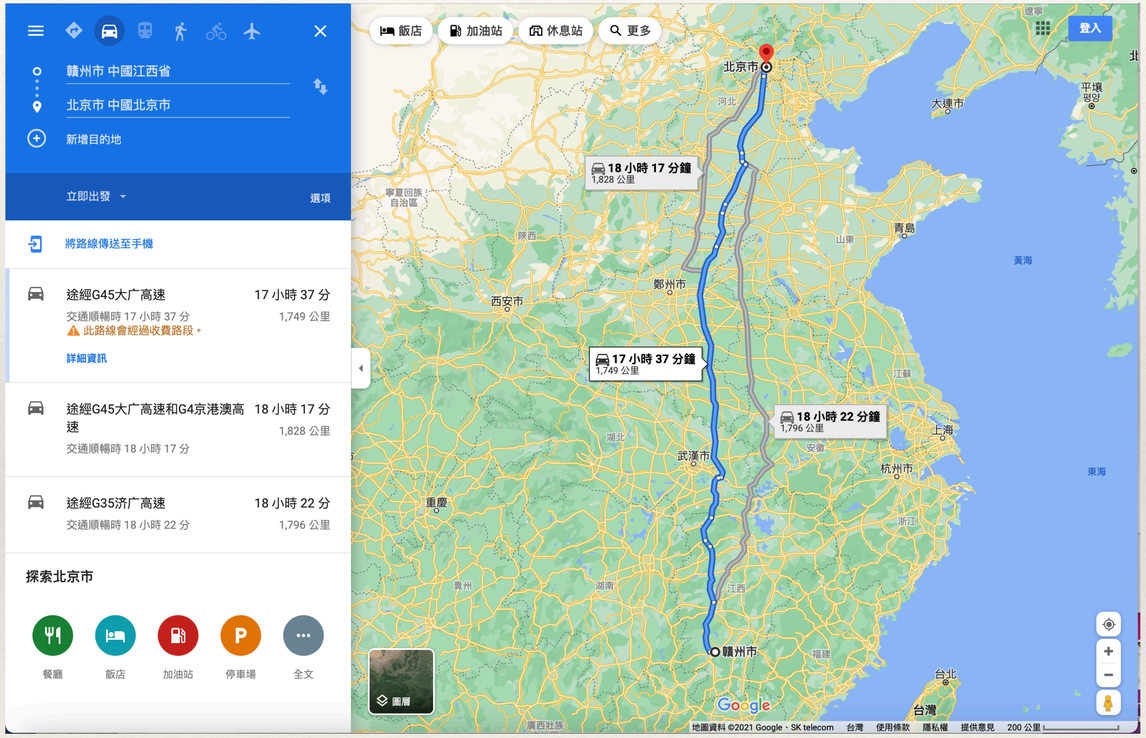
Assumptions:
only 2 choices
with equal prob

Day 1: SZ to Ganzhou

Actual travel time:
5 hours 11 mins



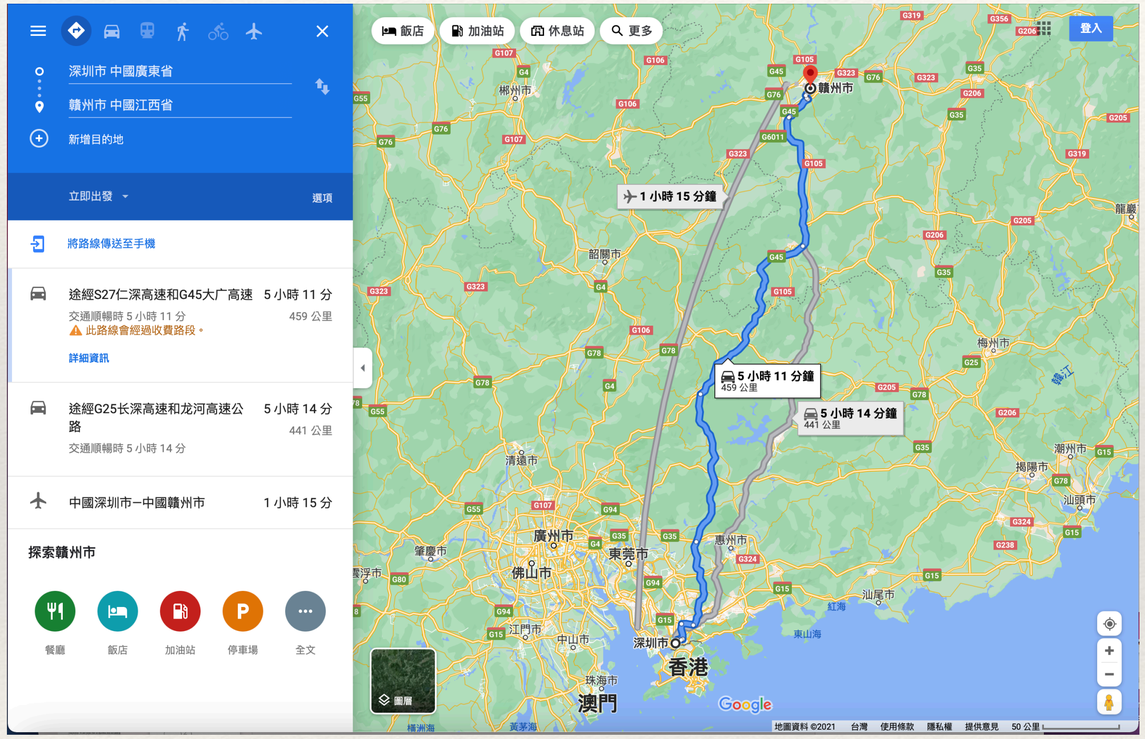
Estimated travel time:
17 hours 37 mins



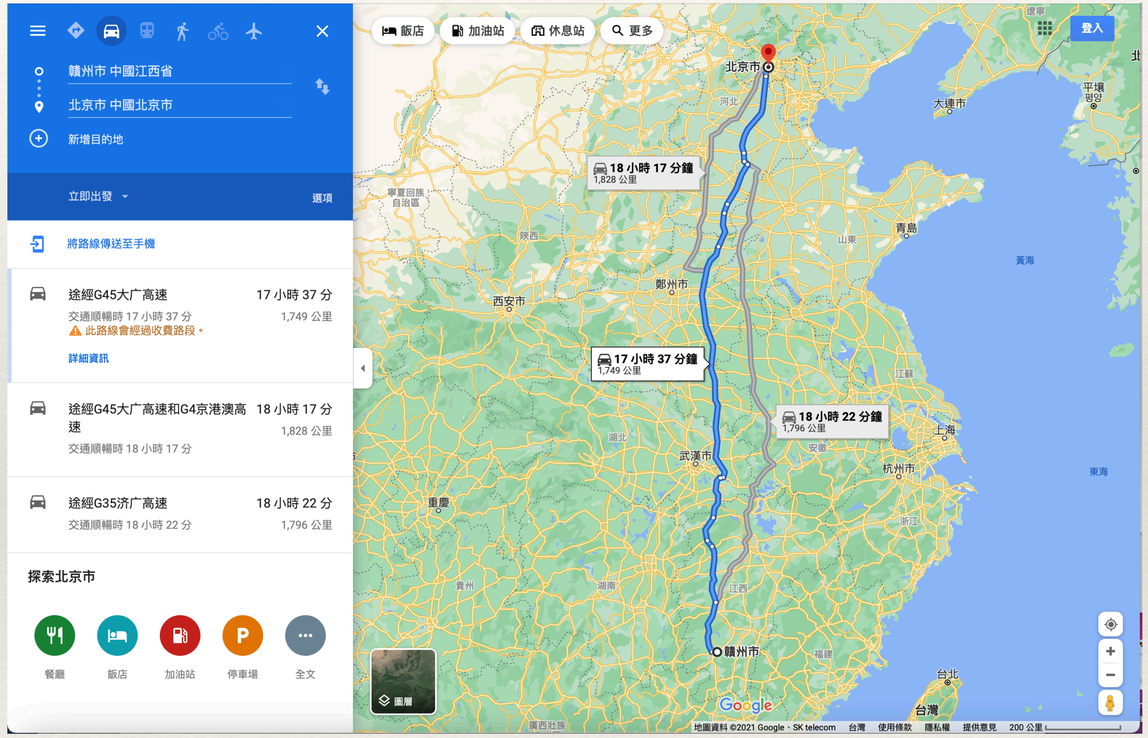
Estimated total travel time at Day 1 :
22 hours 48 mins

Day 1: SZ to Ganzhou

Actual travel time:
5 hours 11 mins



Estimated travel time:
17 hours 37 mins



Estimated total travel time at Day 1 :
22 hours 48 mins

>

Estimated travel time at day 0:
22 hours 26 mins

How to improve policy?

- Consider a deterministic policy, $a = \pi(s)$
- We can *improve* the policy by acting greedily

$$\pi'(s) = \underset{a \in \mathcal{A}}{\operatorname{argmax}}\, q_\pi(s, a)$$

→ Greedy policy

- This improves the value from any state s over one step,

$$q_\pi(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_\pi(s, a) \geq q_\pi(s, \pi(s)) = v_\pi(s)$$

- It therefore improves the value function, $v_{\pi'}(s) \geq v_\pi(s)$

$$\begin{aligned} v_\pi(s) &\leq q_\pi(s, \pi'(s)) = \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s\right] \\ &\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s\right] \\ &\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, \pi'(S_{t+2})) \mid S_t = s\right] \\ &\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \ldots \mid S_t = s\right] = v_{\pi'}(s) \end{aligned}$$

try one step pai‘

v pai 用 q pai逐步替换

Iterative procedures

- Compute state-action value of a policy π_i
  - For s in S and a in A:

$$Q^{\pi_i}(s,a) = R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) V^{\pi_i}(s')$$

- Compute new policy π_{i+1}, for all $s \in S$

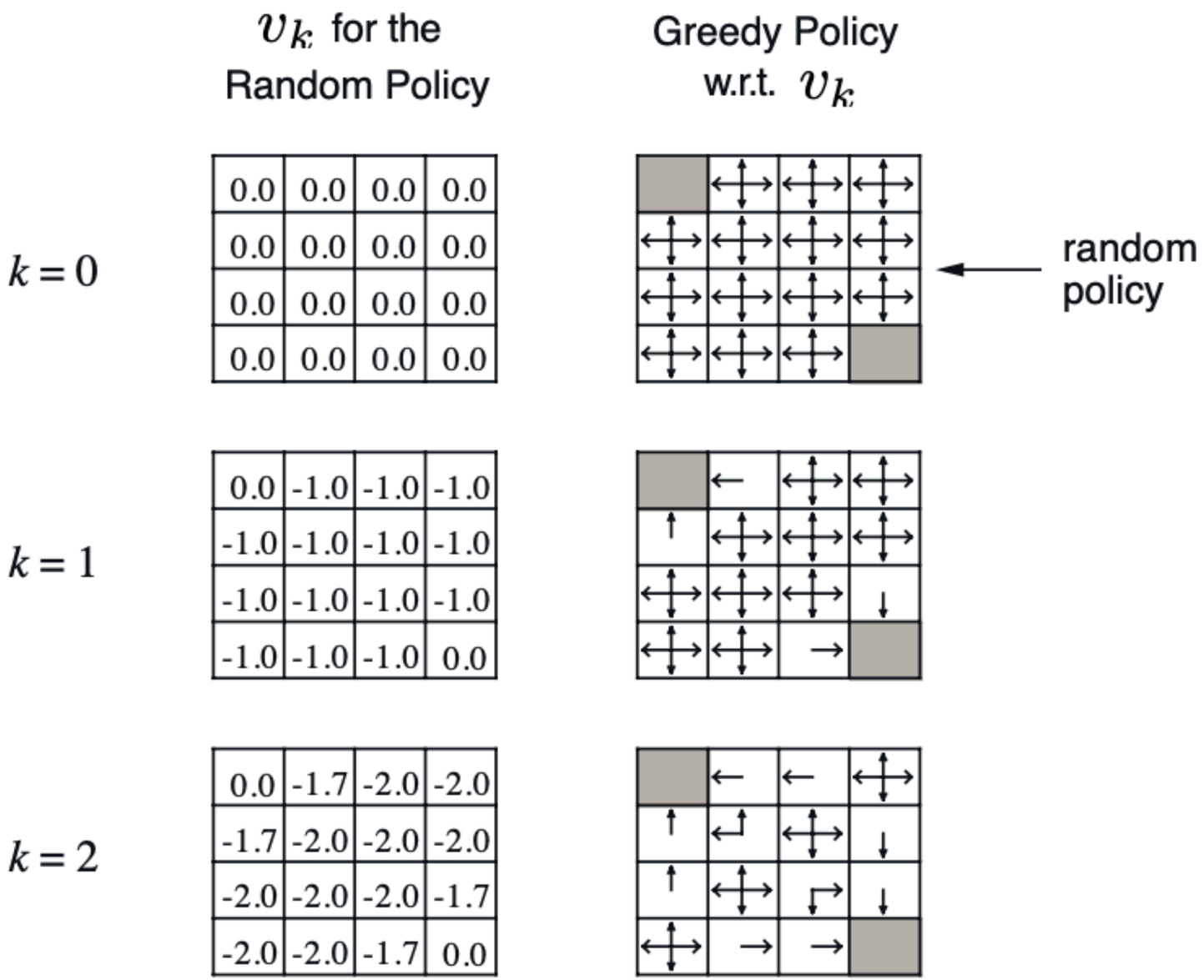
$$\pi_{i+1}(s) = \arg\max_a Q^{\pi_i}(s,a) \ \forall s \in S$$

"Greedy policy"

Algorithm

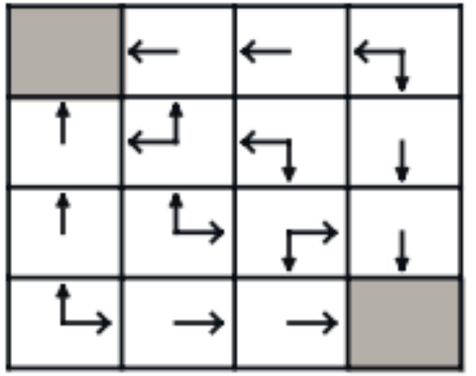
- Set $i = 0$
- Initialize $\pi_0(s)$ randomly for all states s
- While $i == 0$ or $\|\pi_i - \pi_{i-1}\|_1 > 0$ (L1-norm, measures if the policy changed for any state):
  - $V^{\pi_i} \leftarrow$ MDP V function policy **evaluation** of π_i
  - $\pi_{i+1} \leftarrow$ Policy **improvement**
  - $i = i + 1$

Example: gridworld



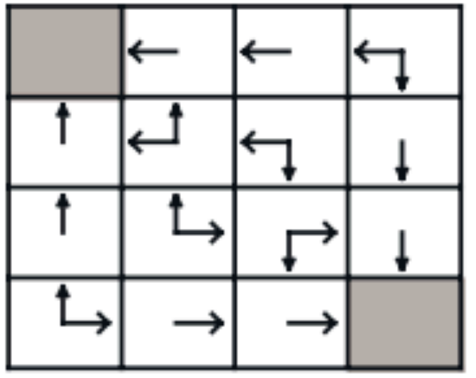
$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0



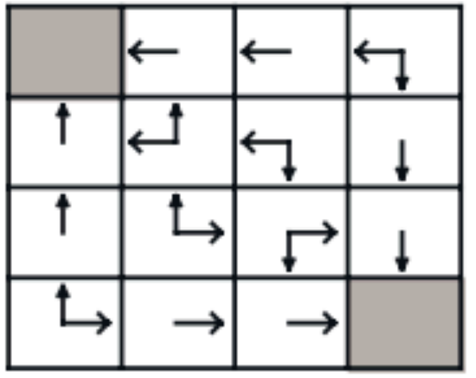
$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0



$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0



optimal policy

Exercise: verify the greedy policy yourself on last 2 pages

Observations

- Policy π_i converges before the value function $v_i(s)$ does
- In small Gridworld, improved policy was optimal
- In general, needing more iterations of improvement / evaluation
- The process of policy iteration always converges to $\pi*$

Exercise: verify the table on last page

Optimal policy found when improvement stops

- If improvements stop,

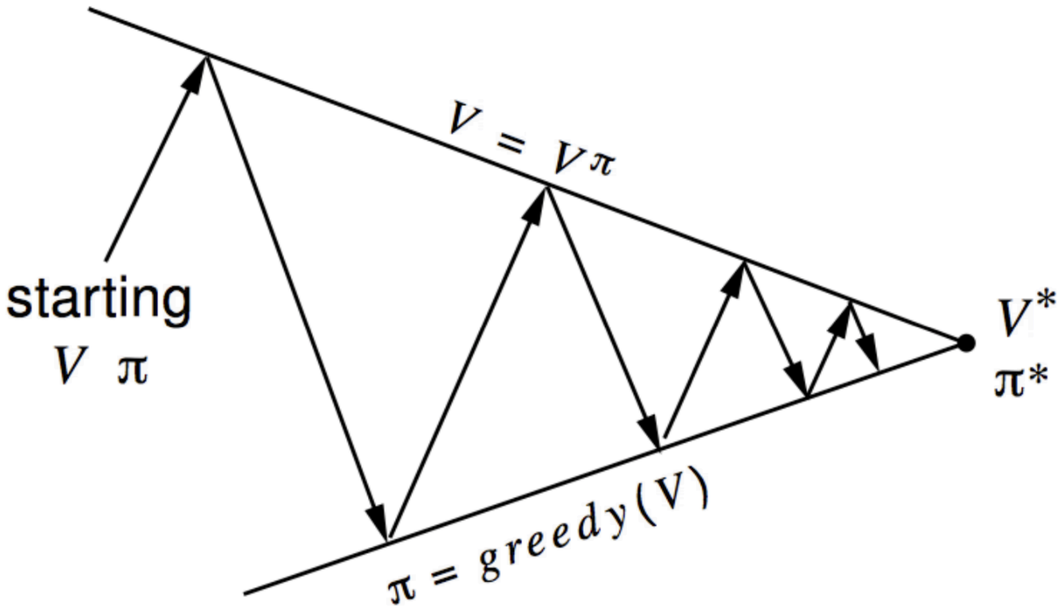
$$q_\pi(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_\pi(s, a) = q_\pi(s, \pi(s)) = v_\pi(s)$$

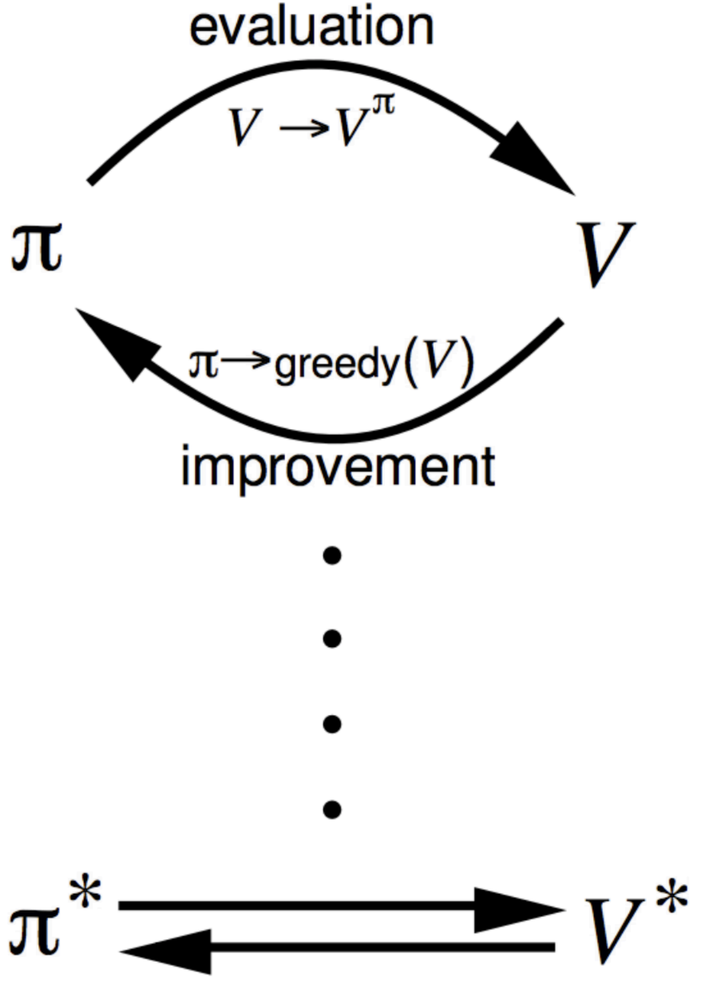
- Then the Bellman optimality equation has been satisfied

$$v_\pi(s) = \max_{a \in \mathcal{A}} q_\pi(s, a)$$

- Therefore $v_\pi(s) = v_*(s)$ for all $s \in \mathcal{S}$
- so π is an optimal policy

Illustration





Policy evaluation Estimate v_π

Iterative policy evaluation

Policy improvement Generate $\pi' \geq \pi$

Greedy policy improvement

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Loop:
      $\Delta \leftarrow 0$
      Loop for each $s \in \mathcal{S}$:
         $v \leftarrow V(s)$
         $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s)) \big[r + \gamma V(s')\big]$
         $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
      old-action $\leftarrow \pi(s)$
      $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a) \big[r + \gamma V(s')\big]$
      If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Example: Jack’s Car Rental

Example 4.2: Jack's Car Rental Jack manages two locations for a nationwide car rental company. Each day, some number of customers arrive at each location to rent cars. If Jack has a car available, he rents it out and is credited \$10 by the national company. If he is out of cars at that location, then the business is lost. Cars become available for renting the day after they are returned. To help ensure that cars are available where they are needed, Jack can move them between the two locations overnight, at a cost of \$2 per car moved. We assume that the number of cars requested and returned at each location are Poisson random variables, meaning that the probability that the number is n is $\frac{\lambda^n}{n!}e^{-\lambda}$, where λ is the expected number. Suppose λ is 3 and 4 for rental requests at the first and second locations and 3 and 2 for returns. To simplify the problem slightly, we assume that there can be no more than 20 cars at each location (any additional cars are returned to the nationwide company, and thus disappear from the problem) and a maximum of five cars can be moved from one location to the other in one night. We take the discount rate to be $\gamma = 0.9$ and formulate this as a continuing finite MDP, where the time steps are days, the state is the number of cars at each location at the end of the day, and the actions are the net numbers of cars moved between the two locations overnight. Figure 4.2 shows the sequence of policies found by policy iteration starting from the policy that never moves any cars.



- States: Two locations, maximum of 20 cars at each
- Actions: Move up to 5 cars between locations overnight
- Reward: \$10 for each car rented (must be available)
- Transitions: Cars returned and requested randomly
  - Poisson distribution, n returns/requests with prob $\frac{\lambda^n}{n!}e^{-\lambda}$
  - 1st location: average requests $= 3$, average returns $= 3$
  - 2nd location: average requests $= 4$, average returns $= 2$

A



Request $\sim$ *Poisson*(3)

Returned $\sim$ *Poisson*(3)

(Pickups $\approx$ returned)

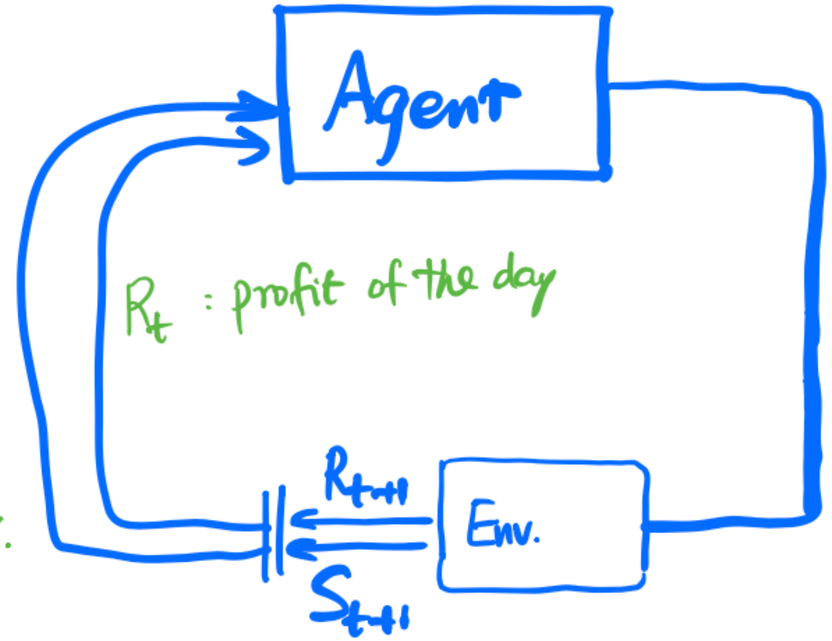
B



Request $\sim$ *Poisson*(4)

Returned $\sim$ *Poisson*(2)

(More pickups than returned)



S_t :
of cars
at ① & ②
at end of day.

R_t : profit of the day

a_t : how many cars
to move from ① → ②
at end of day.

{ demand : requesting cars
supply : returning cars

assumed known
or can be estimated in time.

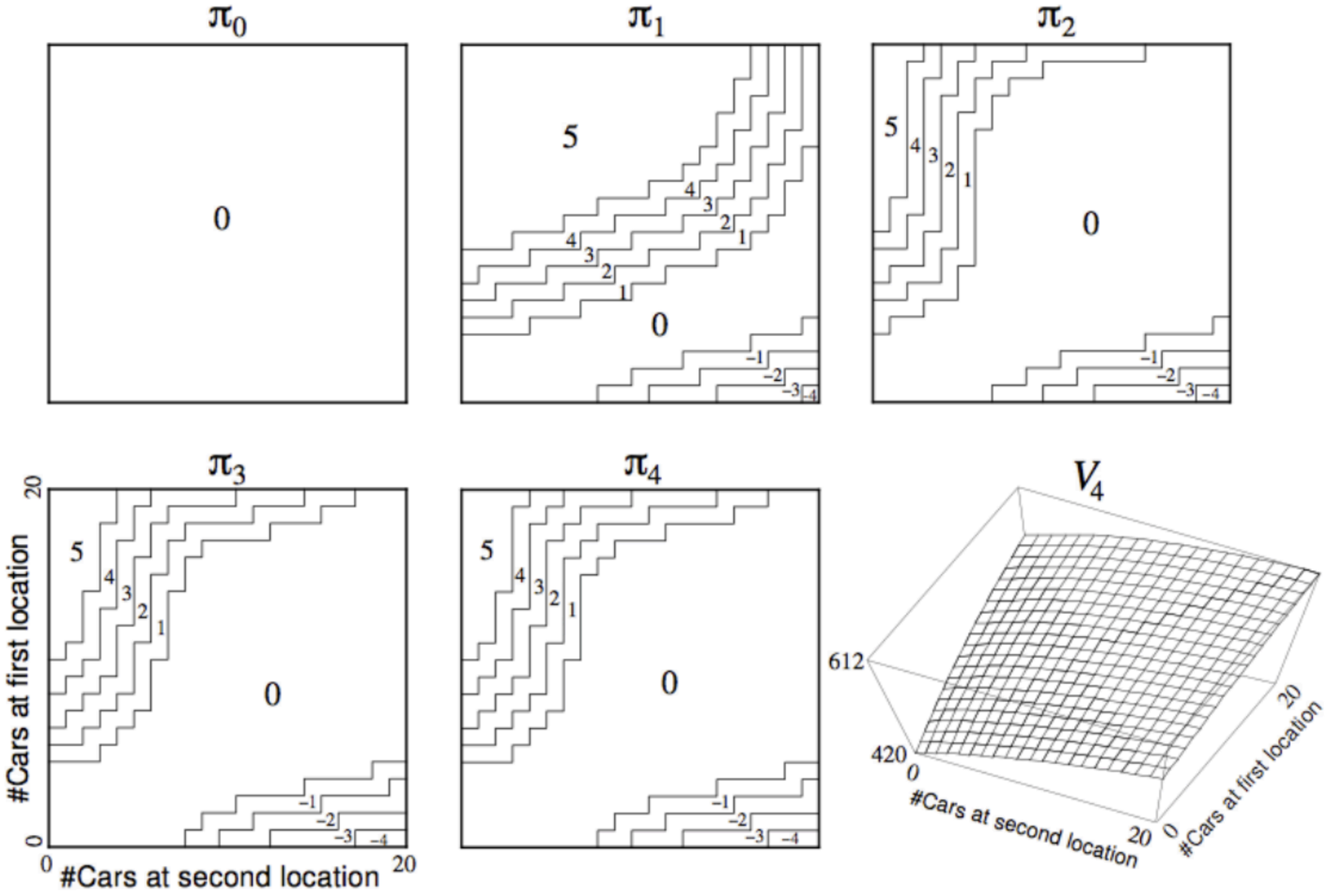
⇓

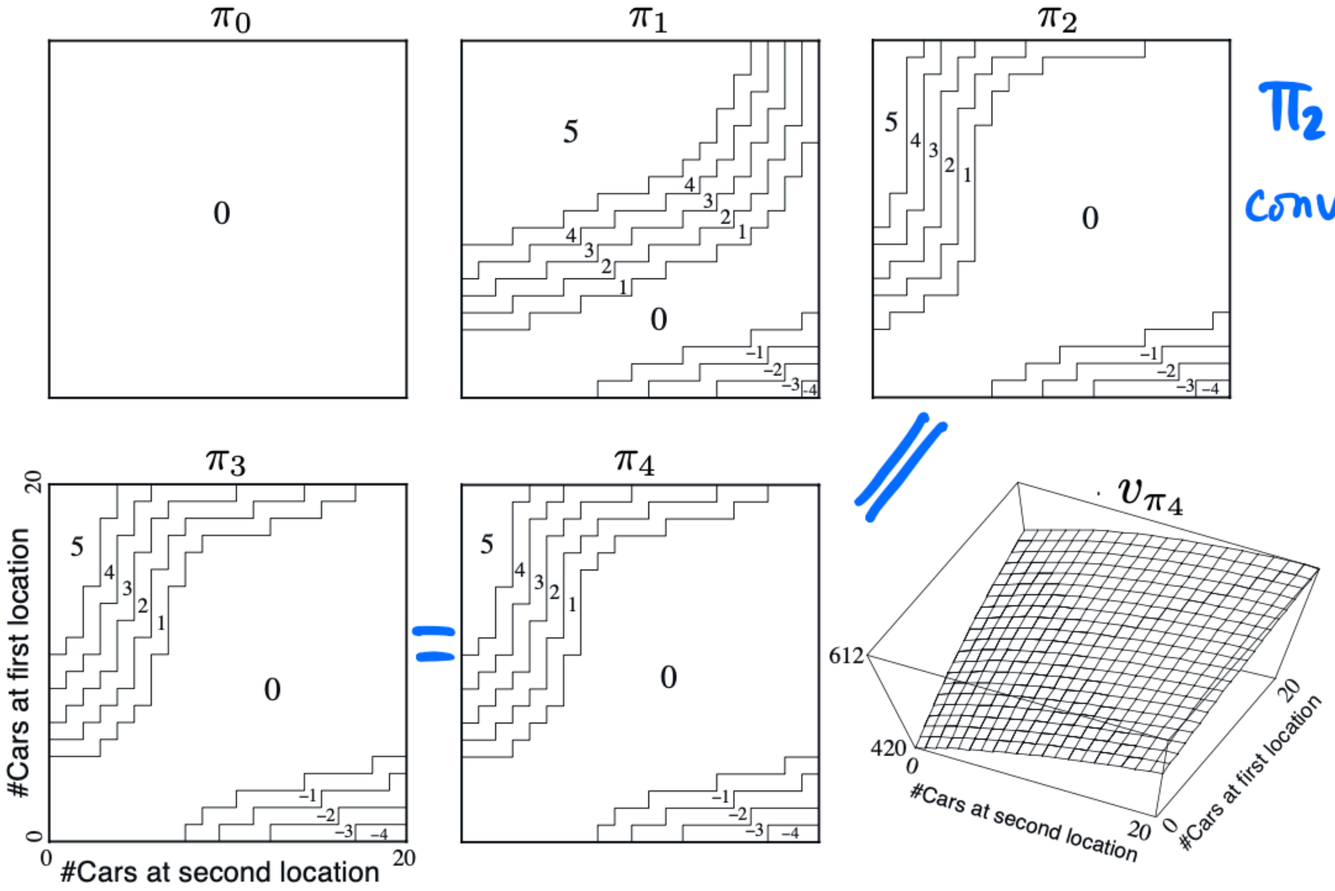
Here Poisson models assumed
with unknown parameters

↓

MLE

Jack's Car Rental: policy iteration





$\pi_2 = \pi_3 = \pi_4$

converges fast

Figure 4.2: The sequence of policies found by policy iteration on Jack's car rental problem, and the final state-value function. The first five diagrams show, for each number of cars at each location at the end of the day, the number of cars to be moved from the first location to the second (negative numbers indicate transfers from the second location to the first). Each successive policy is a strict improvement over the previous policy, and the last policy is optimal. ■

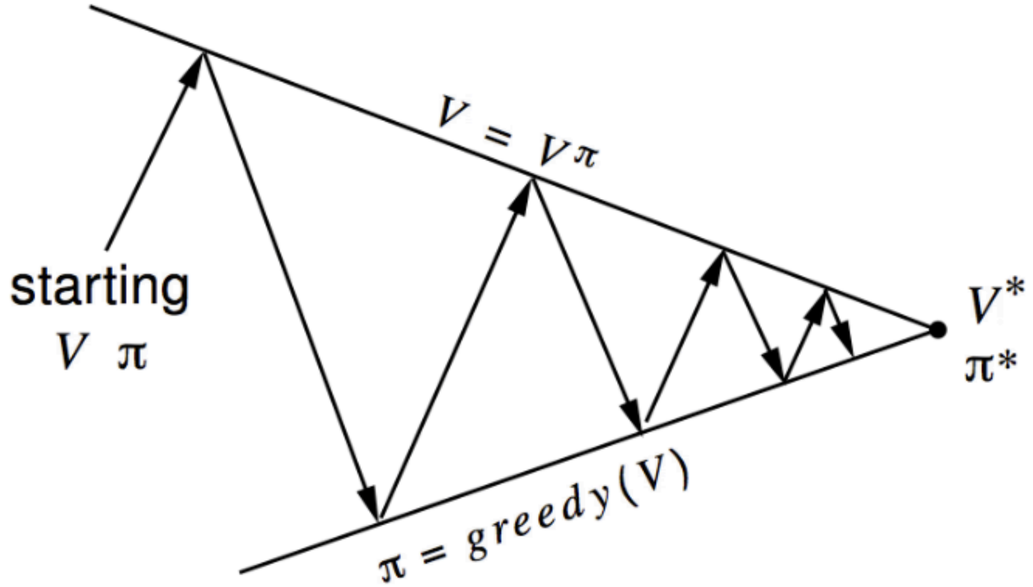
full sample — move to partial sample next class

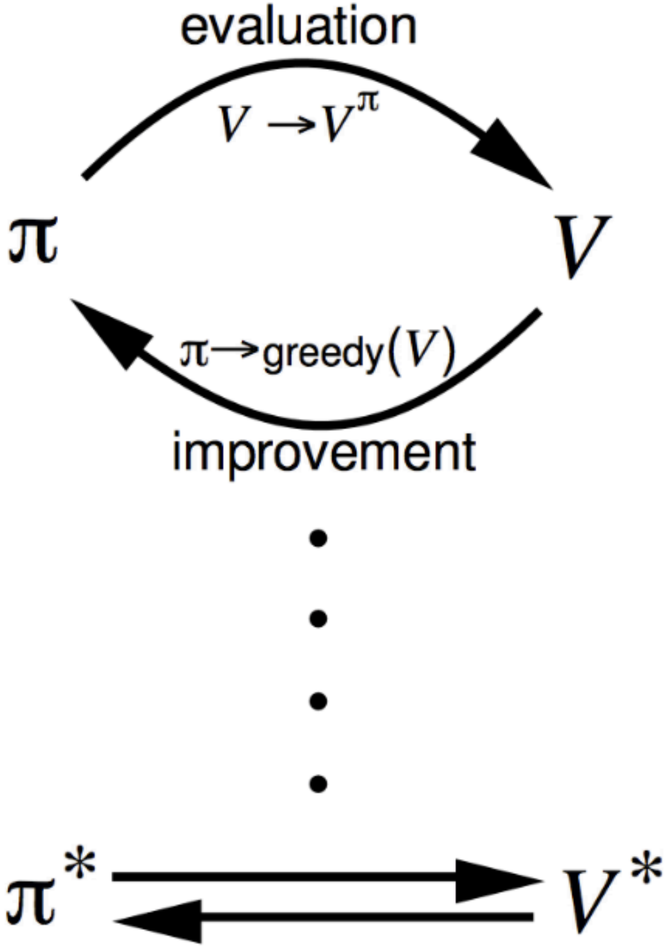
tends to move more cars from ① → ②.

Modified policy iteration

- Does policy evaluation need to converge to v_π?
- Or should we introduce a stopping condition
  - e.g. ϵ-convergence of value function
- Or simply stop after k iterations of iterative policy evaluation?
- For example, in the small gridworld $k = 3$ was sufficient to achieve optimal policy
- Why not update policy every iteration? i.e. stop after $k = 1$
  - This is equivalent to *value iteration* (next section)

Generalised policy iteration





Policy evaluation Estimate v_π

Any policy evaluation algorithm

Policy improvement Generate $\pi' \geq \pi$

Any policy improvement algorithm

Value Iteration

Drawbacks of policy iteration

- Policy evaluation v_π needs many iterations to converge. Slow!!!
- Question: Can we stop policy evaluation after k iterations (even k=1)?
- Answer: Yes

Value evaluation

Truncated policy evaluation + policy improvement

$$
\begin{aligned}
v_{k+1}(s) &\doteq \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a] \\
&= \max_a \sum_{s',r} p(s', r \mid s, a) \Big[r + \gamma v_k(s') \Big],
\end{aligned}
$$

Algorithm

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\mathit{terminal}) = 0$

Loop:
  $\Delta \leftarrow 0$
  Loop for each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
  $\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$

value iteration ←

policy determination ←

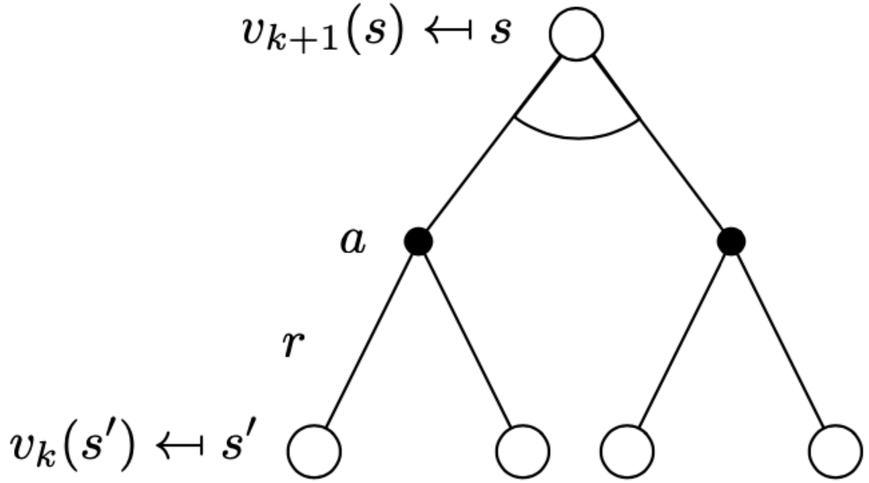
"max"

not "average" (§4.3)

Mini summary on value iteration

- Problem: find optimal policy π
- Solution: iterative application of Bellman optimality backup
- $v_1 \rightarrow v_2 \rightarrow \ldots \rightarrow v_*$
- Using synchronous backups
  - At each iteration $k+1$
  - For all states $s \in \mathcal{S}$
  - Update $v_{k+1}(s)$ from $v_k(s')$
- Convergence to v_* will be proven later
- Unlike policy iteration, there is no explicit policy
- Intermediate value functions may not correspond to any policy

Diagram



$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$

$$\mathbf{v}_{k+1} = \max_{a \in \mathcal{A}} \boldsymbol{\mathcal{R}}^a + \gamma \boldsymbol{\mathcal{P}}^a \mathbf{v}_k$$

Synchronous dynamic programming algorithm

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation + Greedy Policy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

Asynchronous dynamic programming

- DP methods described so far used *synchronous* backups
- i.e. all states are backed up in parallel
- *Asynchronous DP* backs up states individually, in any order
- For each selected state, apply the appropriate backup
- Can significantly reduce computation
- Guaranteed to converge if all states continue to be selected

Three simple ideas

- In-place dynamic programming
- Prioritised sweeping
- Real-time dynamic programming

In-place dynamic programming

- Synchronous value iteration stores two copies of value function

  for all s in $\mathcal{S}$

$$v_{new}(s) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_{old}(s') \right)$$

$$v_{old} \leftarrow v_{new}$$

- In-place value iteration only stores one copy of value function

  for all s in $\mathcal{S}$

$$v(s) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v(s') \right)$$

Prioritised sweeping

- Use magnitude of Bellman error to guide state selection, e.g.

$$\left| \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v(s') \right) - v(s) \right|$$

- Backup the state with the largest remaining Bellman error
- Update Bellman error of affected states after each backup
- Requires knowledge of reverse dynamics (predecessor states)
- Can be implemented efficiently by maintaining a priority queue

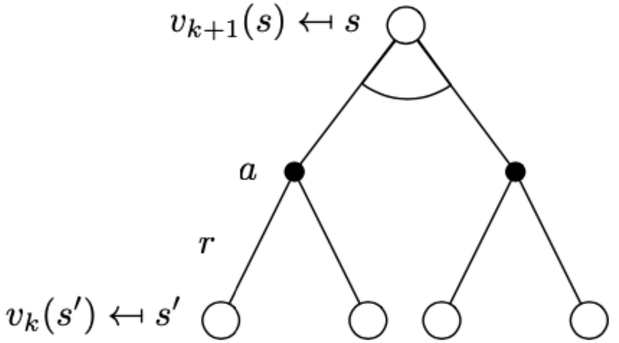
Real-time dynamic programming

- Idea: only states that are relevant to agent
- Use agent's experience to guide the selection of states
- After each time-step S_t, A_t, R_{t+1}
- Backup the state S_t

$$v(S_t) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}^a_{S_t} + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}^a_{S_t s'} v(s') \right)$$

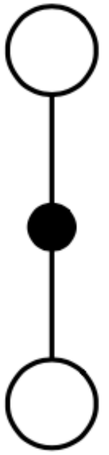
Full-width backups

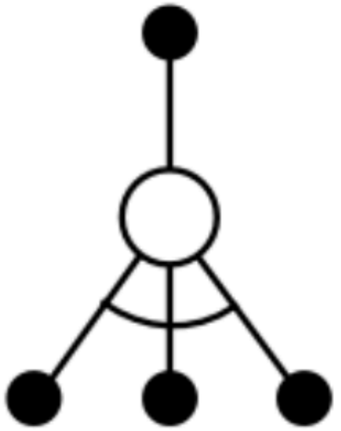
- DP uses *full-width* backups
- For each backup (sync or async)
  - Every successor state and action is considered
  - Using knowledge of the MDP transitions and reward function
- DP is effective for medium-sized problems (millions of states)
- For large problems DP suffers Bellman's *curse of dimensionality*
  - Number of states $n = |\mathcal{S}|$ grows exponentially with number of state variables
- Even one backup can be too expensive



Sample backups

- In subsequent lectures we will consider *sample backups*
- Using sample rewards and sample transitions $\langle S, A, R, S' \rangle$
- Instead of reward function $\mathcal{R}$ and transition dynamics $\mathcal{P}$
- Advantages:
  - Model-free: no advance knowledge of MDP required
  - Breaks the curse of dimensionality through sampling
  - Cost of backup is constant, independent of $n = |\mathcal{S}|$





Approximate dynamic programming

- Approximate the value function
- Using a *function approximator* $\hat{v}(s, \mathbf{w})$
- Apply dynamic programming to $\hat{v}(\cdot, \mathbf{w})$
- e.g. Fitted Value Iteration repeats at each iteration k,
  - Sample states $\tilde{\mathcal{S}} \subseteq \mathcal{S}$
  - For each state $s \in \tilde{\mathcal{S}}$, estimate target value using Bellman optimality equation,

$$\tilde{v}_k(s) = \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \hat{v}(s', \mathbf{w_k}) \right)$$

  - Train next value function $\hat{v}(\cdot, \mathbf{w_{k+1}})$ using targets $\{\langle s, \tilde{v}_k(s) \rangle\}$

Some technical questions

- How do we know that value iteration converges to v_*?
- Or that iterative policy evaluation converges to v_π?
- And therefore that policy iteration converges to v_*?
- Is the solution unique?
- How fast do these algorithms converge?
- These questions are resolved by *contraction mapping theorem*

Value function space

- Consider the vector space $\mathcal{V}$ over value functions
- There are $|\mathcal{S}|$ dimensions
- Each point in this space fully specifies a value function $v(s)$
- What does a Bellman backup do to points in this space?
- We will show that it brings value functions *closer*
- And therefore the backups must converge on a unique solutio

Value function ∞-norm

- We will measure distance between state-value functions u and v by the ∞-norm
- i.e. the largest difference between state values,

$$||u - v||_\infty = \max_{s \in \mathcal{S}} |u(s) - v(s)|$$

Bellman expectation backup is a contraction

- Define the *Bellman expectation backup operator* T^π,

$$T^\pi(v) = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi v$$

- This operator is a γ-contraction, i.e. it makes value functions closer by at least γ,

$$\begin{aligned} ||T^\pi(u) - T^\pi(v)||_\infty &= ||\left(\mathcal{R}^\pi + \gamma \mathcal{P}^\pi u\right) - \left(\mathcal{R}^\pi + \gamma \mathcal{P}^\pi v\right)||_\infty \\ &= ||\gamma \mathcal{P}^\pi (u - v)||_\infty \\ &\leq ||\gamma \mathcal{P}^\pi ||u - v||_\infty||_\infty \\ &\leq \gamma ||u - v||_\infty \end{aligned}$$

Contraction Mapping Theorem

Theorem (Contraction Mapping Theorem)

For any metric space $\mathcal{V}$ that is complete (i.e. closed) under an operator $T(v)$, where T is a γ-contraction,

- *T converges to a unique fixed point*
- *At a linear convergence rate of γ*

Convergence of Iterative Policy Evaluation & Policy Iteration

- The Bellman expectation operator T^π has a unique fixed point
- v_π is a fixed point of T^π (by Bellman expectation equation)
- By contraction mapping theorem
- Iterative policy evaluation converges on v_π
- Policy iteration converges on v_*

Bellman Optimality Backup is a Contraction

- Define the *Bellman optimality backup operator* T^*,

$$T^*(v) = \max_{a \in \mathcal{A}} \mathcal{R}^a + \gamma \mathcal{P}^a v$$

- This operator is a γ-contraction, i.e. it makes value functions closer by at least γ (similar to previous proof)

$$||T^*(u) - T^*(v)||_\infty \leq \gamma ||u - v||_\infty$$

Convergence of Value Function

- The Bellman optimality operator T^* has a unique fixed point
- v_* is a fixed point of T^* (by Bellman optimality equation)
- By contraction mapping theorem
- Value iteration converges on v_*

